Exponential Families for Bandit Models

The Natural Language of Modern Bandit Models

Author: Tang

Institute: Tang's Machine Learning Blog

Date: 2026-06-20

Version: 1.0.0

Topic: Exponential Families, KL-UCB, Thompson Sampling

Contents

Chapter 1 Three Reward Models, One Repeated Calculation

A bandit can return very different kinds of rewards.

A recommendation is clicked or ignored. A sensor reports a real number with measurement noise. A server receives a count of requests during the next second. The first observation is binary, the second is continuous, and the third is a nonnegative integer.

At first these look like three unrelated statistical problems. Yet after a few observations, the learner keeps asking the same questions:

- What single number summarizes what this arm has shown so far?
- Which parameter value makes those observations most plausible?
- How quickly can a wrong parameter value be ruled out?
- How should a prior belief be updated after one more reward?

For Bernoulli, Gaussian, and Poisson rewards, the algebra behind all four questions has the same shape. That shared shape is the exponential family.

The name can make the subject sound more exotic than it is. The central idea is modest:

Put the unknown parameter next to a summary of the data, and place everything needed for normalization in one convex function.

We will first see the pattern inside familiar distributions. Only then will we give it a name.

1.1 Binary rewards: the data become a count

Let $X \in \{0, 1\}$ and let p be the probability of a success. Then

$$\mathbb{P}_p(X = x) = p^x(1-p)^{1-x}.$$

For observations $x_1, \dots, x_n$,

$$\begin{aligned} L(p; x_1, \dots, x_n) &= \prod_{i=1}^{n} p^{x_i}(1-p)^{1-x_i} \\ &= p^{\sum_{i=1}^n x_i}(1-p)^{n-\sum_{i=1}^n x_i}. \end{aligned}$$

Define

$$S_n = \sum_{i=1}^{n} x_i.$$

Then

$$L(p; x_1, \dots, x_n) = p^{S_n}(1-p)^{n-S_n}.$$

The order of clicks and non-clicks has disappeared. The likelihood remembers only how many clicks occurred.

1.2 Gaussian rewards: the data become a sum

Suppose $X \sim \mathcal{N}(\mu, \sigma^2)$ and the variance σ^2 is known. Its density is

$$p_\mu(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}.$$

For n independent observations,

$$\begin{aligned}\log L(\mu) &= -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i-\mu)^2 \\ &= -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}x_i^2 + \frac{\mu}{\sigma^2}\sum_{i=1}^{n}x_i - \frac{n\mu^2}{2\sigma^2}.\end{aligned}$$

As a function of μ, the sample enters through

$$S_n = \sum_{i=1}^{n} x_i.$$

Again, the full history can be compressed to a running total.

1.3 Count rewards: the data become a sum again

Suppose $X \sim \text{Poisson}(\lambda)$. Then

$$\mathbb{P}_\lambda(X = x) = \frac{e^{-\lambda}\lambda^x}{x!}, \qquad x = 0, 1, 2, \ldots.$$

For n independent observations,

$$\begin{aligned}L(\lambda) &= \prod_{i=1}^{n}\frac{e^{-\lambda}\lambda^{x_i}}{x_i!} \\ &= \frac{1}{\prod_{i=1}^{n} x_i!}\exp\left\{-n\lambda + \left(\sum_{i=1}^{n} x_i\right)\log\lambda\right\}.\end{aligned}$$

The parameter-dependent part again uses only

$$S_n = \sum_{i=1}^{n} x_i.$$

Key idea

The exponential-family idea is already visible. A potentially long dataset is replaced by a short running summary. The model then compares that summary with what each parameter value predicts.

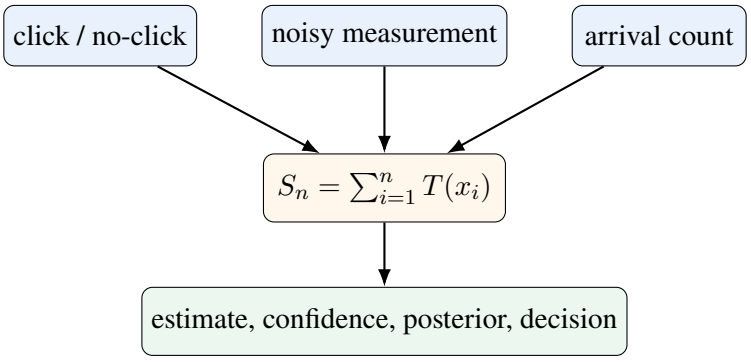


Figure 1.1: Different observations can feed the same statistical workflow.

Chapter 2 The Template Appears

A one-parameter exponential family has probability mass or density

$$\boxed{p_\eta(x) = h(x)\exp\{\eta T(x) - A(\eta)\}.} \tag{2.1}$$

The notation is easiest to understand one piece at a time.

- x is the observation.
- $T(x)$ is the part of the observation that talks directly to the parameter. It is called the sufficient statistic for one observation.
- η is the natural parameter.
- $h(x)$ contains everything that depends on x but not on η.
- $A(\eta)$ is the log-partition function. It makes the total probability equal to one.

The word "natural" does not mean that η is always the parameter a practitioner would report. A practitioner reports a click probability p, a mean μ, or a rate λ. The natural parameter is the coordinate that makes the log-likelihood linear in the data summary.

Taking logs in (2.1),

$$\log p_\eta(x) = \log h(x) + \eta T(x) - A(\eta).$$

The unknown parameter and the observed statistic meet through the simple product

$$\eta T(x).$$

Everything else has a separate job.

Think

Why not place an arbitrary constant in $A(\eta)$? Because changing A changes the total mass of the density. Once h, T, and the reference measure are fixed, normalization determines A.

Chapter 3 The Log-Partition Function Is the Engine

Let

$$Z(\eta) = \int h(x) e^{\eta T(x)} \, \mathrm{d}\nu(x),$$

where the integral means a sum for a discrete model and an ordinary integral for a continuous model. To make (2.1) integrate to one, we need

$$\begin{aligned} 1 &= \int p_\eta(x) \, \mathrm{d}\nu(x) \\ &= \int h(x) e^{\eta T(x) - A(\eta)} \, \mathrm{d}\nu(x) \\ &= e^{-A(\eta)} \int h(x) e^{\eta T(x)} \, \mathrm{d}\nu(x) \\ &= e^{-A(\eta)} Z(\eta). \end{aligned}$$

Therefore

$$e^{A(\eta)} = Z(\eta),$$

and hence

$$\boxed{A(\eta) = \log \int h(x) e^{\eta T(x)} \, \mathrm{d}\nu(x).} \tag{3.1}$$

At first, A looks like a normalization term that we must carry around. In fact, it stores almost everything we want to know.

3.1 First derivative: the mean

Write

$$Z(\eta) = e^{A(\eta)}.$$

Differentiate $A(\eta) = \log Z(\eta)$:

$$\begin{aligned} A'(\eta) &= \frac{Z'(\eta)}{Z(\eta)} \\ &= \frac{\int T(x) h(x) e^{\eta T(x)} \, \mathrm{d}\nu(x)}{\int h(x) e^{\eta T(x)} \, \mathrm{d}\nu(x)} \\ &= \int T(x) h(x) e^{\eta T(x) - A(\eta)} \, \mathrm{d}\nu(x) \\ &= \int T(x) p_\eta(x) \, \mathrm{d}\nu(x) \\ &= \mathbb{E}_\eta[T(X)]. \end{aligned}$$

Thus

$$\boxed{A'(\eta) = \mathbb{E}_\eta[T(X)].} \tag{3.2}$$

The derivative of the normalizer is the mean of the statistic.

3.2 Second derivative: the variance

Differentiate once more:

$$\begin{aligned} A''(\eta) &= \frac{Z''(\eta)Z(\eta) - [Z'(\eta)]^2}{[Z(\eta)]^2} \\ &= \frac{Z''(\eta)}{Z(\eta)} - \left(\frac{Z'(\eta)}{Z(\eta)}\right)^2 \\ &= \mathbb{E}_\eta[T(X)^2] - \mathbb{E}_\eta[T(X)]^2 \\ &= \mathrm{Var}_\eta(T(X)). \end{aligned}$$

Therefore

$$\boxed{A''(\eta) = \mathrm{Var}_\eta(T(X)) \geq 0.} \tag{3.3}$$

So A is convex. This convexity is not a decorative theorem. It gives uniqueness of the mean map, concavity of the likelihood, the local form of KL divergence, and the curvature used in concentration bounds.

Result

For a regular one-parameter exponential family,

$$A(\eta) \quad \longrightarrow \quad A'(\eta) = \text{mean statistic} \quad \longrightarrow \quad A''(\eta) = \text{variance of the statistic.}$$

One function stores normalization, location, and uncertainty.

3.3 The vector version

If the statistic and natural parameter are vectors,

$$p_{\boldsymbol{\eta}}(x) = h(x)\exp\{\boldsymbol{\eta}^\top \boldsymbol{T}(x) - A(\boldsymbol{\eta})\},$$

then the same calculation gives

$$\nabla A(\boldsymbol{\eta}) = \mathbb{E}_{\boldsymbol{\eta}}[\boldsymbol{T}(X)]$$

and

$$\nabla^2 A(\boldsymbol{\eta}) = \mathrm{Cov}_{\boldsymbol{\eta}}(\boldsymbol{T}(X)).$$

The Hessian is positive semidefinite because every covariance matrix is positive semidefinite.

The one-dimensional case is enough for the present bandit chapter, but the vector identity is one reason exponential families sit at the center of graphical models and variational inference [6, 12].

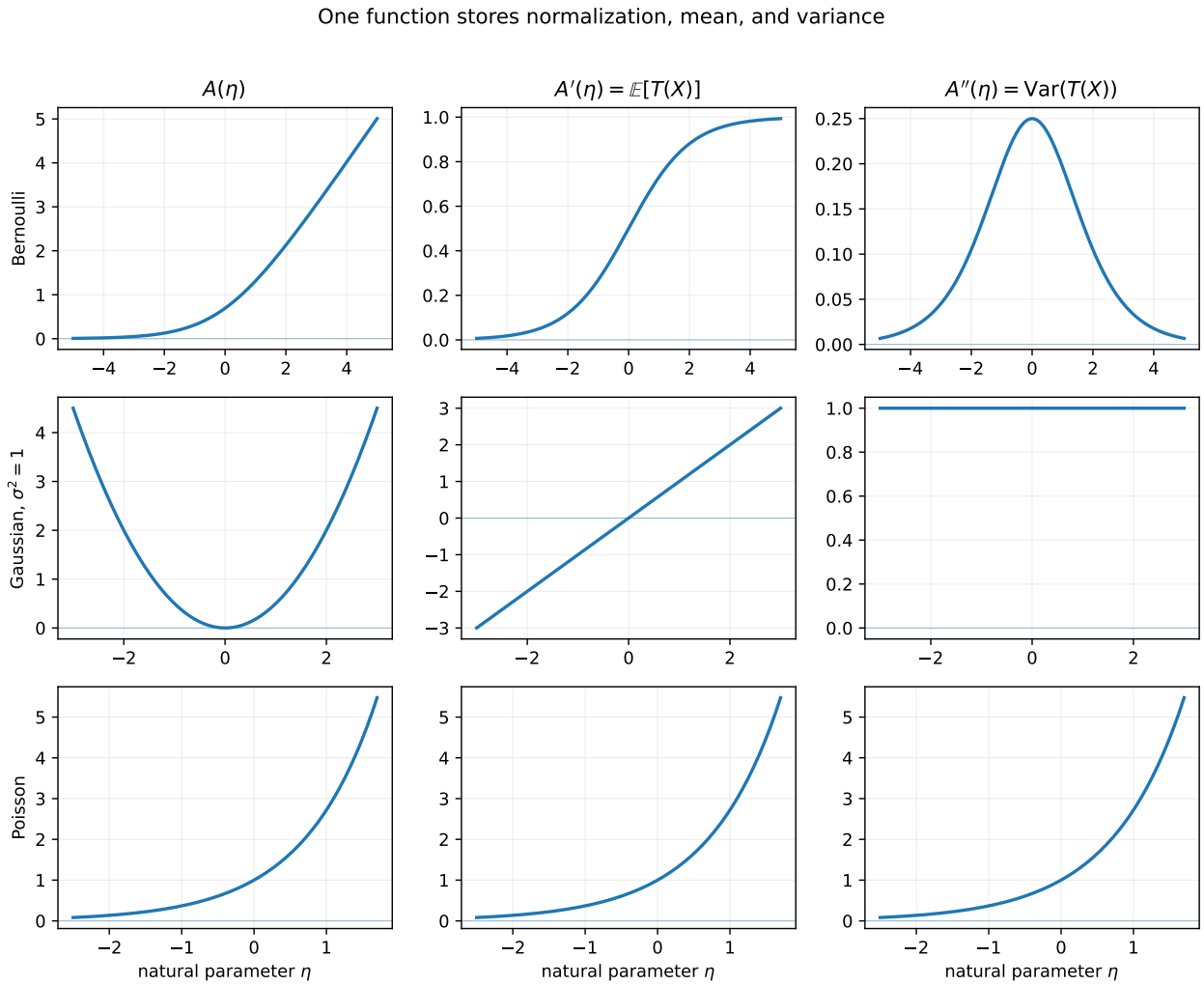


Figure 3.1: The same derivative identities hold for three familiar reward families.

Chapter 4 Three Canonical Examples, Derived Slowly

4.1 Bernoulli rewards

Start from

$$p_p(x) = p^x(1-p)^{1-x}, \qquad x \in \{0,1\}.$$

Take logs inside the exponential:

$$\begin{aligned} p_p(x) &= \exp\{x \log p + (1-x)\log(1-p)\} \\ &= \exp\{x \log p + \log(1-p) - x\log(1-p)\} \\ &= \exp\left\{x \log \frac{p}{1-p} + \log(1-p)\right\}. \end{aligned}$$

Define the natural parameter

$$\eta = \log \frac{p}{1-p}.$$

Solving for p,

$$\begin{aligned} e^\eta &= \frac{p}{1-p}, \\ e^\eta(1-p) &= p, \\ e^\eta &= p(1+e^\eta), \\ p &= \frac{e^\eta}{1+e^\eta}. \end{aligned}$$

Also,

$$1 - p = \frac{1}{1+e^\eta},$$

so

$$\log(1-p) = -\log(1+e^\eta).$$

Therefore

$$p_\eta(x) = \exp\{\eta x - \log(1+e^\eta)\}.$$

We can now read off

$$T(x) = x, \qquad h(x) = 1, \qquad A(\eta) = \log(1+e^\eta).$$

Differentiate:

$$\begin{aligned} A'(\eta) &= \frac{e^\eta}{1+e^\eta} \\ &= p, \end{aligned}$$

and

$$\begin{aligned} A''(\eta) &= \frac{e^\eta(1+e^\eta) - e^{2\eta}}{(1+e^\eta)^2} \\ &= \frac{e^\eta}{(1+e^\eta)^2} \\ &= p(1-p). \end{aligned}$$

The derivative identities reproduce the Bernoulli mean and variance.

4.2 Gaussian rewards with known variance

Let

$$p_\mu(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(x-\mu)^2}{2\sigma^2}\right\}.$$

Expand the square:

$$\begin{aligned} -\frac{(x-\mu)^2}{2\sigma^2} &= -\frac{x^2 - 2\mu x + \mu^2}{2\sigma^2} \\ &= -\frac{x^2}{2\sigma^2} + \frac{\mu x}{\sigma^2} - \frac{\mu^2}{2\sigma^2}. \end{aligned}$$

Hence

$$p_\mu(x) = \underbrace{\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{x^2}{2\sigma^2}\right\}}_{h(x)} \exp\left\{\frac{\mu}{\sigma^2}x - \frac{\mu^2}{2\sigma^2}\right\}.$$

Set

$$\eta = \frac{\mu}{\sigma^2}.$$

Then

$$\mu = \sigma^2\eta$$

and

$$\frac{\mu^2}{2\sigma^2} = \frac{\sigma^2\eta^2}{2}.$$

Thus

$$p_\eta(x) = h(x) \exp\left\{\eta x - \frac{\sigma^2\eta^2}{2}\right\},$$

so

$$T(x) = x, \qquad A(\eta) = \frac{\sigma^2\eta^2}{2}.$$

Differentiate:

$$A'(\eta) = \sigma^2\eta = \mu$$

and

$$A''(\eta) = \sigma^2.$$

Again the mean and variance are recovered immediately.

4.3 Poisson rewards

Start from

$$p_\lambda(x) = \frac{e^{-\lambda}\lambda^x}{x!}.$$

Write it as one exponential:

$$p_\lambda(x) = \frac{1}{x!} \exp\{-\lambda + x\log\lambda\}.$$

Set

$$\eta = \log\lambda, \qquad \lambda = e^\eta.$$

Then

$$p_\eta(x) = \frac{1}{x!}\exp\{\eta x - e^\eta\}.$$

Therefore

$$T(x) = x, \qquad h(x) = \frac{1}{x!}, \qquad A(\eta) = e^\eta.$$

Differentiate:

$$A'(\eta) = e^\eta = \lambda$$

and

$$A''(\eta) = e^\eta = \lambda.$$

The Poisson mean and variance are both equal to the rate.

4.4 A compact dictionary

Table 4.1: Canonical forms used frequently in bandit models.

Family	Natural parameter	$T(x)$	$A(\eta)$	Mean $A'(\eta)$
Bernoulli	$\eta = \log[p/(1-p)]$	x	$\log(1+e^\eta)$	$e^\eta/(1+e^\eta)$
Gaussian, known σ^2	$\eta = \mu/\sigma^2$	x	$\sigma^2\eta^2/2$	$\sigma^2\eta$
Poisson	$\eta = \log\lambda$	x	e^η	e^η
Exponential	$\eta = -\lambda < 0$	x	$-\log(-\eta)$	$-1/\eta$
Gamma, fixed shape α	$\eta = -\beta < 0$	x	$-\alpha\log(-\eta)$	$-\alpha/\eta$

The family is broad enough to cover many reward models, but not every parameterization of every distribution is a one-parameter exponential family. The support must not change with the parameter, and the parameter must enter in the canonical linear form.

Chapter 5 A Dataset Collapses to a Running Statistic

Let $X_1, \dots, X_n$ be independent observations from (2.1). Their joint density is

$$p_\eta(x_1, \dots, x_n) = \prod_{i=1}^{n} h(x_i) e^{\eta T(x_i) - A(\eta)}$$
$$= \left(\prod_{i=1}^{n} h(x_i) \right) \exp\left\{ \eta \sum_{i=1}^{n} T(x_i) - nA(\eta) \right\}.$$

Define

$$S_n = \sum_{i=1}^{n} T(X_i).$$

Then

$$\boxed{p_\eta(x_1, \dots, x_n) = H(x_1, \dots, x_n) \exp\{\eta S_n - nA(\eta)\}.} \tag{5.1}$$

Once n and S_n are known, the rest of the sample no longer changes the likelihood ratio between two parameter values. This is the operational meaning of sufficiency here.

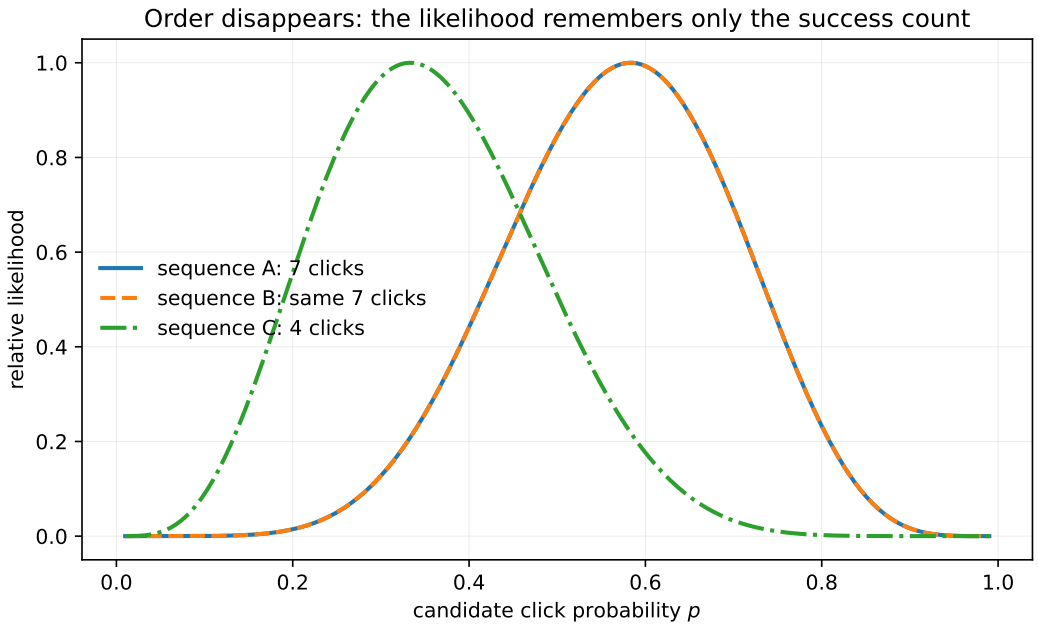


Figure 5.1: Two Bernoulli sequences with the same number of successes produce exactly the same likelihood curve.

5.1 Maximum likelihood becomes moment matching

Ignoring terms that do not depend on η, the log-likelihood is

$$\ell_n(\eta) = \eta S_n - nA(\eta).$$

Differentiate:

$$\ell_n'(\eta) = S_n - nA'(\eta).$$

At an interior maximum,

$$0 = S_n - nA'(\widehat{\eta}_n),$$
$$A'(\widehat{\eta}_n) = \frac{S_n}{n}.$$

But $A'(\eta) = \mathbb{E}_\eta[T(X)]$. Therefore

$$\boxed{\mathbb{E}_{\widehat{\eta}_n}[T(X)] = \frac{1}{n}\sum_{i=1}^{n} T(X_i).} \tag{5.2}$$

The fitted model chooses the parameter whose expected statistic matches the observed average statistic.

The second derivative is

$$\begin{aligned}\ell_n''(\eta) &= -nA''(\eta) \\ &= -n\,\mathrm{Var}_\eta(T(X)) \\ &\le 0.\end{aligned}$$

So the log-likelihood is concave. In a regular nondegenerate family, the stationary point is the unique maximum.

5.2 The familiar estimators fall out immediately

For Bernoulli rewards,

$$A'(\eta) = p, \qquad T(X) = X,$$

so

$$\widehat{p}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

For Gaussian rewards with known variance,

$$A'(\eta) = \mu, \qquad T(X) = X,$$

so

$$\widehat{\mu}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

For Poisson rewards,

$$A'(\eta) = \lambda, \qquad T(X) = X,$$

so

$$\widehat{\lambda}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

These estimators look identical because the three models use the same statistic $T(X) = X$. Their uncertainty is not identical; that difference is stored in A''.

Research lens

The reusable state of a one-parameter exponential-family arm is often just two numbers:

$$N_a(t) \quad \text{and} \quad S_a(t) = \sum_{s \le t : A_s = a} T(X_s).$$

This is more than a coding convenience. It is the exact likelihood state needed by maximum likelihood,

KL-UCB, and conjugate Bayesian updates.

Chapter 6 Conjugate Priors Are Closed Bookkeeping Rules

A Bayesian learner begins with a prior and multiplies it by the likelihood. A conjugate prior is chosen so that this multiplication changes only a small set of hyperparameters.

For the canonical family, consider a prior on η of the form

$$\pi(\eta \mid \xi, \nu) \propto \exp\{\xi\eta - \nu A(\eta)\}\mathbf{1}\{\eta \in \mathcal{H}\}. \tag{6.1}$$

Here $\mathcal{H}$ is the natural-parameter space. The constants ξ and ν must be chosen so that the prior is integrable.

After observing $x_1, \ldots, x_n$, Bayes' rule gives

$$\begin{aligned}\pi(\eta \mid x_1, \ldots, x_n) &\propto \pi(\eta) \prod_{i=1}^{n} p_\eta(x_i) \\ &\propto \exp\{\xi\eta - \nu A(\eta)\} \exp\{\eta S_n - nA(\eta)\} \\ &= \exp\{(\xi + S_n)\eta - (\nu + n)A(\eta)\}.\end{aligned}$$

Thus

$$\boxed{\xi \leftarrow \xi + S_n, \qquad \nu \leftarrow \nu + n.} \tag{6.2}$$

The posterior has the same form as the prior. The old pseudo-total and new total are added; the old pseudo-count and new count are added.

6.1 Beta-Bernoulli, including the change of coordinates

For Bernoulli rewards,

$$\eta = \log \frac{p}{1-p}, \qquad A(\eta) = \log(1 + e^\eta).$$

The canonical prior in η is

$$\pi_\eta(\eta) \propto e^{\xi\eta - \nu A(\eta)}.$$

Since

$$p = \frac{e^\eta}{1 + e^\eta}, \qquad \frac{\mathrm{d}\eta}{\mathrm{d}p} = \frac{1}{p(1-p)},$$

the induced density in p is

$$\begin{aligned}\pi_p(p) &= \pi_\eta(\eta(p)) \left|\frac{\mathrm{d}\eta}{\mathrm{d}p}\right| \\ &\propto \exp\left\{\xi \log \frac{p}{1-p} - \nu \log \frac{1}{1-p}\right\} \frac{1}{p(1-p)} \\ &= p^{\xi-1}(1-p)^{\nu-\xi-1}.\end{aligned}$$

This is a Beta distribution with

$$\alpha = \xi, \qquad \beta = \nu - \xi.$$

If S_n successes are observed in n trials, then

$$\alpha \leftarrow \alpha + S_n, \qquad \beta \leftarrow \beta + n - S_n.$$

6.2 Normal-Normal with known observation variance

For $X \sim \mathcal{N}(\mu, \sigma^2)$,

$$\eta = \frac{\mu}{\sigma^2}, \qquad A(\eta) = \frac{\sigma^2 \eta^2}{2}.$$

The generic prior is

$$\begin{aligned}\pi(\eta) &\propto \exp\left\{\xi\eta - \frac{\nu\sigma^2\eta^2}{2}\right\} \\ &\propto \exp\left\{-\frac{\nu\sigma^2}{2}\left(\eta - \frac{\xi}{\nu\sigma^2}\right)^2\right\}.\end{aligned}$$

Hence

$$\eta \sim \mathcal{N}\left(\frac{\xi}{\nu\sigma^2}, \frac{1}{\nu\sigma^2}\right).$$

Because $\mu = \sigma^2\eta$,

$$\mu \sim \mathcal{N}\left(\frac{\xi}{\nu}, \frac{\sigma^2}{\nu}\right).$$

After n observations,

$$\mu \mid X_{1:n} \sim \mathcal{N}\left(\frac{\xi + \sum_i X_i}{\nu + n}, \frac{\sigma^2}{\nu + n}\right).$$

The posterior mean is a weighted average:

$$\frac{\xi + \sum_i X_i}{\nu + n} = \frac{\nu}{\nu + n}\frac{\xi}{\nu} + \frac{n}{\nu + n}\frac{1}{n}\sum_i X_i.$$

6.3 Gamma-Poisson

For Poisson rewards,

$$\eta = \log\lambda, \qquad A(\eta) = e^\eta.$$

The prior in η is

$$\pi_\eta(\eta) \propto e^{\xi\eta - \nu e^\eta}.$$

Since $\lambda = e^\eta$ and $\mathrm{d}\eta/\mathrm{d}\lambda = 1/\lambda$,

$$\begin{aligned}\pi_\lambda(\lambda) &\propto \lambda^\xi e^{-\nu\lambda}\frac{1}{\lambda} \\ &= \lambda^{\xi-1}e^{-\nu\lambda}.\end{aligned}$$

This is a Gamma distribution with shape ξ and rate ν. After observing counts $X_1, \ldots, X_n$,

$$\xi \leftarrow \xi + \sum_{i=1}^n X_i, \qquad \nu \leftarrow \nu + n.$$

Key idea

Conjugacy is not a mysterious Bayesian coincidence. The likelihood contributes $\eta S_n - nA(\eta)$, so a prior built from the same two terms can absorb new data by addition.

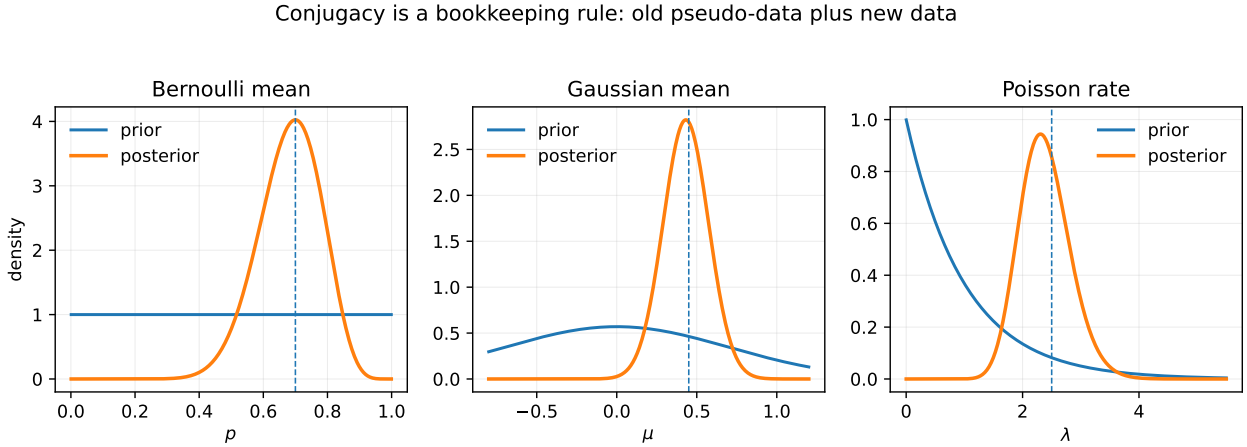


Figure 6.1: The posterior keeps the same shape because sufficient statistics add. Dashed lines mark empirical means.

Chapter 7 KL Divergence Becomes Convex Geometry

The previous chapter treated KL divergence as average log-evidence. Inside an exponential family, it has a closed form.

Take two members of the same family, P_η and P_ζ. The log-likelihood ratio is

$$\begin{aligned}\log \frac{p_\eta(x)}{p_\zeta(x)} &= \log \frac{h(x)e^{\eta T(x)-A(\eta)}}{h(x)e^{\zeta T(x)-A(\zeta)}} \\ &= (\eta-\zeta)T(x) - A(\eta) + A(\zeta).\end{aligned}$$

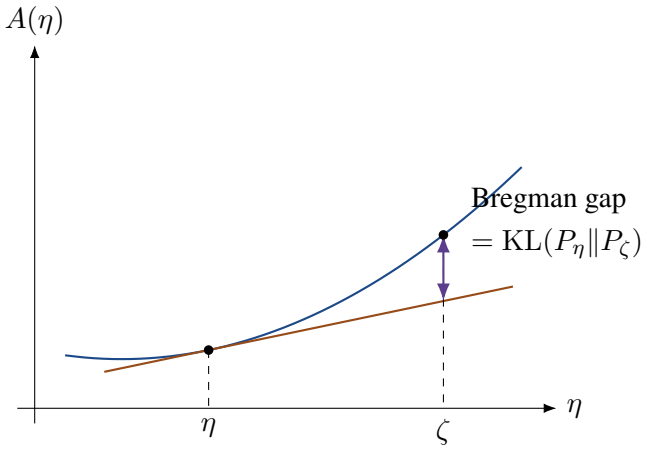
Take expectation under P_η:

$$\begin{aligned}\mathrm{KL}(P_\eta \| P_\zeta) &= (\eta-\zeta)\mathbb{E}_\eta[T(X)] - A(\eta) + A(\zeta) \\ &= (\eta-\zeta)A'(\eta) - A(\eta) + A(\zeta) \\ &= A(\zeta) - A(\eta) - A'(\eta)(\zeta-\eta).\end{aligned}$$

Therefore

$$\boxed{\mathrm{KL}(P_\eta \| P_\zeta) = A(\zeta) - A(\eta) - A'(\eta)(\zeta-\eta).} \tag{7.1}$$

The right side is the gap between the convex function $A(\zeta)$ and the tangent line to A at η. Convexity makes the gap nonnegative.



7.1 The local quadratic and Fisher information

Let $\zeta = \eta + h$. Taylor expand $A(\eta + h)$:

$$A(\eta+h) = A(\eta) + A'(\eta)h + \frac{1}{2}A''(\eta)h^2 + O(h^3).$$

Insert this into (7.1):

$$\begin{aligned}\mathrm{KL}(P_\eta \| P_{\eta+h}) &= A(\eta+h) - A(\eta) - A'(\eta)h \\ &= \frac{1}{2}A''(\eta)h^2 + O(h^3).\end{aligned}$$

The score is

$$\frac{\partial}{\partial \eta}\log p_\eta(X) = T(X) - A'(\eta).$$

Therefore the Fisher information is

$$\begin{aligned} I(\eta) &= \mathbb{E}_\eta\left[\left(\frac{\partial}{\partial\eta}\log p_\eta(X)\right)^2\right] \\ &= \mathbb{E}_\eta[(T(X) - A'(\eta))^2] \\ &= \mathrm{Var}_\eta(T(X)) \\ &= A''(\eta). \end{aligned}$$

Thus

$$\boxed{\mathrm{KL}(P_\eta \| P_{\eta+h}) = \frac{1}{2} I(\eta) h^2 + O(h^3).} \tag{7.2}$$

7.2 The mean coordinate and the convex dual

Define the mean parameter

$$\mu = A'(\eta).$$

The convex conjugate of A is

$$A^*(\mu) = \sup_\eta \{\eta\mu - A(\eta)\}.$$

At the matching pair $\mu = A'(\eta)$,

$$A^*(\mu) = \eta\mu - A(\eta).$$

Let $\mu_\eta = A'(\eta)$ and $\mu_\zeta = A'(\zeta)$. The Bregman divergence generated by A^* is

$$\begin{aligned} D_{A^*}(\mu_\eta, \mu_\zeta) &= A^*(\mu_\eta) - A^*(\mu_\zeta) - \zeta(\mu_\eta - \mu_\zeta) \\ &= [\eta\mu_\eta - A(\eta)] - [\zeta\mu_\zeta - A(\zeta)] - \zeta\mu_\eta + \zeta\mu_\zeta \\ &= (\eta - \zeta)\mu_\eta - A(\eta) + A(\zeta) \\ &= \mathrm{KL}(P_\eta \| P_\zeta). \end{aligned}$$

So KL is a Bregman divergence in either coordinate system, with the order changing through convex duality. This link underlies a broad connection between exponential families and Bregman geometry [1, 12].

Chapter 8 Concentration Comes from the Same Function

The moment-generating function of $T(X)$ is almost free once A is known. For any λ such that $\eta + \lambda$ remains in the natural-parameter space,

$$\begin{aligned}\mathbb{E}_\eta[e^{\lambda T(X)}] &= \int e^{\lambda T(x)} h(x) e^{\eta T(x) - A(\eta)} \, \mathrm{d}\nu(x) \\ &= e^{-A(\eta)} \int h(x) e^{(\eta+\lambda)T(x)} \, \mathrm{d}\nu(x) \\ &= e^{-A(\eta)} e^{A(\eta+\lambda)} \\ &= \exp\{A(\eta+\lambda) - A(\eta)\}.\end{aligned}$$

Hence, with $\mu = A'(\eta)$,

$$\boxed{\mathbb{E}_\eta[e^{\lambda(T(X)-\mu)}] = \exp\{A(\eta+\lambda) - A(\eta) - \lambda\mu\}.} \tag{8.1}$$

This identity turns the log-partition function into a concentration engine.

8.1 Chernoff's method in the family

Let $T_1, \dots, T_n$ be independent copies of $T(X)$, and let

$$\overline{T}_n = \frac{1}{n} \sum_{i=1}^{n} T_i.$$

For $x > \mu$ and $\lambda > 0$,

$$\begin{aligned}\mathbb{P}_\eta(\overline{T}_n \geq x) &= \mathbb{P}_\eta\left(e^{\lambda \sum_i T_i} \geq e^{\lambda n x}\right) \\ &\leq e^{-\lambda n x} \mathbb{E}_\eta\left[e^{\lambda \sum_i T_i}\right] \\ &= e^{-\lambda n x} \prod_{i=1}^{n} \mathbb{E}_\eta[e^{\lambda T_i}] \\ &= \exp\{-n[\lambda x - A(\eta+\lambda) + A(\eta)]\}.\end{aligned}$$

We choose the best λ:

$$\mathbb{P}_\eta(\overline{T}_n \geq x) \leq \exp\left\{-n \sup_{\lambda > 0} [\lambda x - A(\eta+\lambda) + A(\eta)]\right\}.$$

Let η_x satisfy

$$A'(\eta_x) = x.$$

Differentiate the expression inside the supremum:

$$\frac{\mathrm{d}}{\mathrm{d}\lambda}[\lambda x - A(\eta+\lambda) + A(\eta)] = x - A'(\eta+\lambda).$$

The optimum is reached at

$$\eta + \lambda^* = \eta_x, \qquad \lambda^* = \eta_x - \eta.$$

Substitute:

$$\begin{aligned}\lambda^* x - A(\eta+\lambda^*) + A(\eta) &= (\eta_x - \eta)x - A(\eta_x) + A(\eta) \\ &= (\eta_x - \eta)A'(\eta_x) - A(\eta_x) + A(\eta) \\ &= \mathrm{KL}(P_{\eta_x} \| P_\eta).\end{aligned}$$

Therefore

$$\boxed{\mathbb{P}_\eta(\overline{T}_n \geq x) \leq \exp\{-n\,\mathrm{KL}(P_{\eta_x} \| P_\eta)\}.} \tag{8.2}$$

The same KL divergence that measures evidence also controls rare sample means.

Proof pattern

A recurring proof pattern in bandit theory is

log-partition $\longrightarrow$ moment-generating function $\longrightarrow$ convex optimization $\longrightarrow$ KL exponent.

Once the family is identified, many concentration calculations become instances of this one pattern.

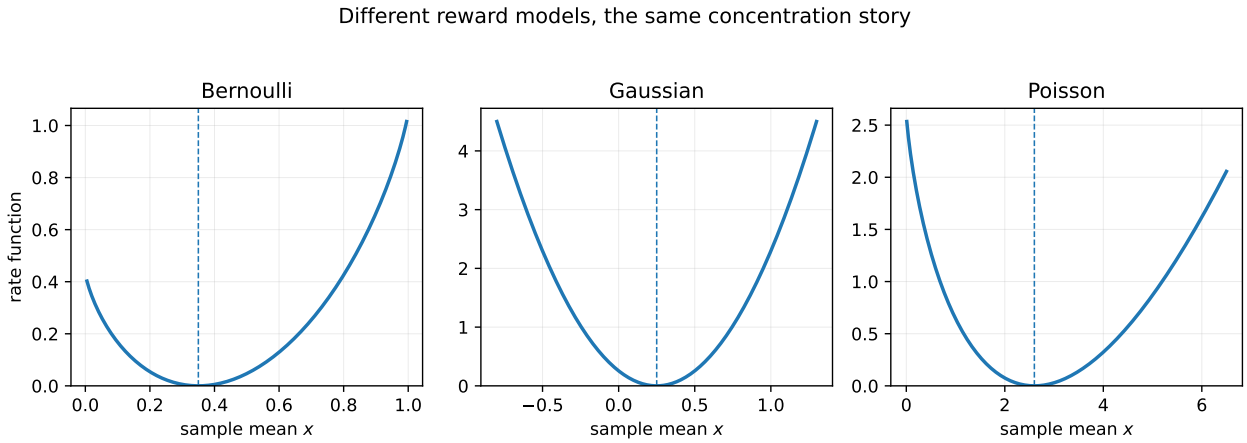


Figure 8.1: The rate functions differ in shape, but all arise by the same exponential-family calculation.

Chapter 9 An Exponential-Family Bandit Keeps One Ledger per Arm

Consider K arms. Arm a has parameter η_a and mean

$$\mu_a = A'(\eta_a).$$

At round t, the learner chooses A_t using the past and observes a reward X_t from the chosen arm.

The adaptive choice of arms changes which observations are collected. It does not change the form of the reward likelihood. Up to terms independent of the parameters,

$$\begin{aligned}\log p_{\boldsymbol{\eta}}(H_T) &= \sum_{t=1}^{T} [\eta_{A_t} T(X_t) - A(\eta_{A_t})] + \text{constant} \\ &= \sum_{a=1}^{K} \left[\eta_a \sum_{t=1}^{T} \mathbf{1}\{A_t = a\} T(X_t) - A(\eta_a) \sum_{t=1}^{T} \mathbf{1}\{A_t = a\} \right] + extconstant.\end{aligned}$$

Define

$$N_a(T) = \sum_{t=1}^{T} \mathbf{1}\{A_t = a\}$$

and

$$S_a(T) = \sum_{t=1}^{T} \mathbf{1}\{A_t = a\} T(X_t).$$

Then

$$\boxed{\log p_{\boldsymbol{\eta}}(H_T) = \sum_{a=1}^{K} [\eta_a S_a(T) - N_a(T) A(\eta_a)] + \text{constant}.} \tag{9.1}$$

Every arm needs a count and a sufficient-statistic total. The algorithm decides where the next line is written; the exponential family decides how the ledger is interpreted.

9.1 The empirical mean parameter

For arm a, the maximum-likelihood equation is

$$A'(\widehat{\eta}_a) = \frac{S_a}{N_a}.$$

Define

$$\widehat{\mu}_a = \frac{S_a}{N_a}.$$

When $T(X) = X$, this is the ordinary sample mean. The meaning of its uncertainty still depends on the reward family.

9.2 Distribution-aware optimism

Let

$$d(\mu, q) = \mathrm{KL}(P_\mu \| P_q)$$

be the KL divergence written in mean coordinates. A generic KL upper confidence index is

$$U_a(t) = \sup \{q : \ N_a(t) d(\widehat{\mu}_a(t), q) \le \beta(t)\}. \tag{9.2}$$

The policy chooses

$$A_t \in \arg\max_a U_a(t).$$

The same line of code means different confidence shapes in different families:

Table 9.1: KL divergence between two mean parameters.

Family	$d(\mu, q) = \mathrm{KL}(P_\mu \| P_q)$
Bernoulli	$\mu \log(\mu/q) + (1-\mu)\log[(1-\mu)/(1-q)]$
Gaussian, known σ^2	$(\mu - q)^2/(2\sigma^2)$
Poisson	$\mu \log(\mu/q) + q - \mu$
Exponential, mean μ	$\log(q/\mu) + \mu/q - 1$

For the Gaussian family, (9.2) can be solved directly:

$$\begin{aligned} N_a \frac{(\widehat{\mu}_a - q)^2}{2\sigma^2} &\le \beta(t), \\ (q - \widehat{\mu}_a)^2 &\le \frac{2\sigma^2 \beta(t)}{N_a}, \\ q &\le \widehat{\mu}_a + \sqrt{\frac{2\sigma^2 \beta(t)}{N_a}}. \end{aligned}$$

Thus

$$U_a(t) = \widehat{\mu}_a(t) + \sqrt{\frac{2\sigma^2 \beta(t)}{N_a(t)}}.$$

For Bernoulli and Poisson rewards, a one-dimensional binary search finds the endpoint.

9.3 Posterior sampling uses the same ledger

With independent conjugate priors, the posterior of arm a updates as

$$\xi_a(t) = \xi_{a,0} + S_a(t), \qquad \nu_a(t) = \nu_{a,0} + N_a(t).$$

Thompson sampling draws one parameter from each posterior and acts as if the draw were true:

$$\begin{aligned} \widetilde{\eta}_a(t) &\sim \pi_a(\eta_a \mid H_{t-1}), \\ A_t &\in \arg\max_a A'(\widetilde{\eta}_a(t)). \end{aligned}$$

For Bernoulli, this is Beta sampling. For Gaussian rewards with known variance, it is Normal sampling. For Poisson rewards, it is Gamma sampling.

Exponential-family structure has therefore supported both optimistic and posterior-sampling analyses. Korda, Kaufmann, and Munos used its closed-form KL and Fisher information to analyze Thompson sampling in one-dimensional families [8]. Jin, Xu, Xiao, and Anandkumar later developed finite-time and asymptotic guarantees for exponential-family bandits through ExpTS and ExpTS$^+$ [7]. The point of the abstraction is not merely to shorten notation; it lets one proof strategy travel across reward models.

Research lens

A useful research question is often not “Which named distribution should I use?” but:

What are the sufficient statistic, log-partition function, mean map, KL geometry, and conjugate update of the reward model?

Once these five objects are known, much of the bandit machinery becomes visible.

Chapter 10 From Mathematics to an Algorithm Interface

The implementation can mirror the theory. A model only needs to provide a few operations:

- convert accumulated statistics into an empirical mean;
- compute or invert the family KL divergence;
- update and sample from a conjugate posterior;
- generate rewards for simulation.

The decision loop then stays almost unchanged.

```
def kl_ucb_step(model, counts, totals, t):
    empirical_mean = totals / counts
    beta = np.log(t) + 3.0 * np.log(max(np.log(t), 1.0))
    upper_index = model.kl_upper(empirical_mean, counts, beta)
    return int(np.argmax(upper_index))
```

For Gaussian rewards, the model-specific inversion is closed form:

```
def gaussian_kl_upper(empirical_mean, counts, beta, sigma):
    return empirical_mean + np.sqrt(
        2.0 * sigma**2 * beta / counts
    )
```

For Bernoulli or Poisson rewards, monotonicity makes binary search sufficient:

```
def kl_upper_by_bisection(empirical_mean, counts, beta,
                          kl_divergence, upper_bracket,
                          steps=24):
    lo = empirical_mean.copy()
    hi = upper_bracket(empirical_mean, counts, beta)

    for _ in range(steps):
        mid = (lo + hi) / 2.0
        feasible = counts * kl_divergence(empirical_mean, mid) <= beta
        lo = np.where(feasible, mid, lo)
        hi = np.where(feasible, hi, mid)

    return lo
```

The Thompson-sampling skeleton is even shorter:

```
def thompson_step(model, counts, totals, rng):
    sampled_means = model.sample_posterior_mean(
        counts=counts,
        totals=totals,
```

```
        rng=rng,
    )
    return int(np.argmax(sampled_means))
```

The abstraction is useful only if the model methods remain mathematically transparent. A large software hierarchy that hides the likelihood would defeat the purpose.

Chapter 11 Reproducible Experiment: One Skeleton, Three Families

The supplied script builds three four-armed environments:

- Bernoulli means $(0.18, 0.24, 0.31, 0.36)$;
- Gaussian means $(0.00, 0.10, 0.18, 0.25)$ with known $\sigma = 0.35$;
- Poisson means $(1.70, 2.00, 2.30, 2.60)$.

For each environment, it runs a family-aware KL-UCB policy and conjugate Thompson sampling for $T = 5000$ rounds over 240 independent runs. The performance measure is pseudo-regret,

$$R_T = \sum_{t=1}^{T} (\mu^* - \mu_{A_t}).$$

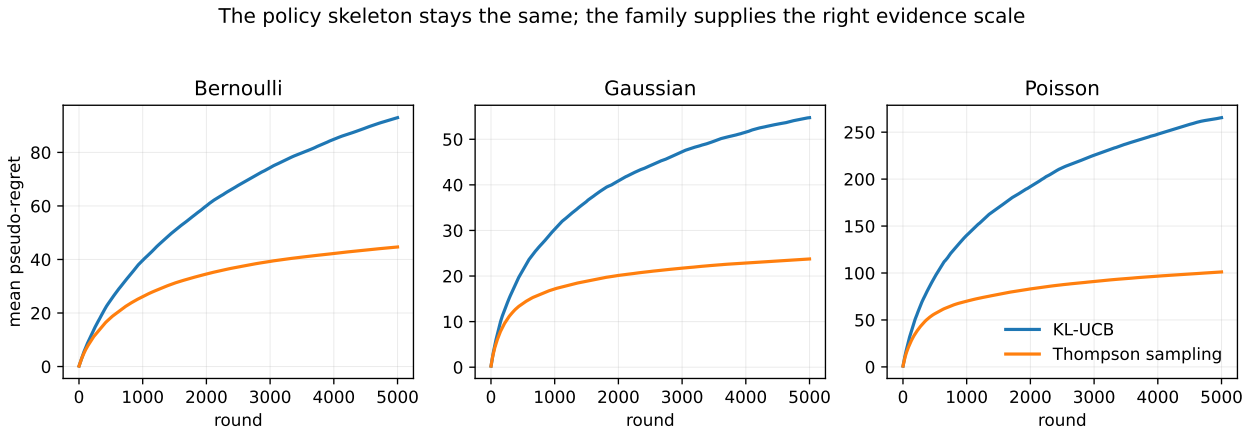


Figure 11.1: The algorithmic skeleton is unchanged; the family determines the KL and posterior operations.

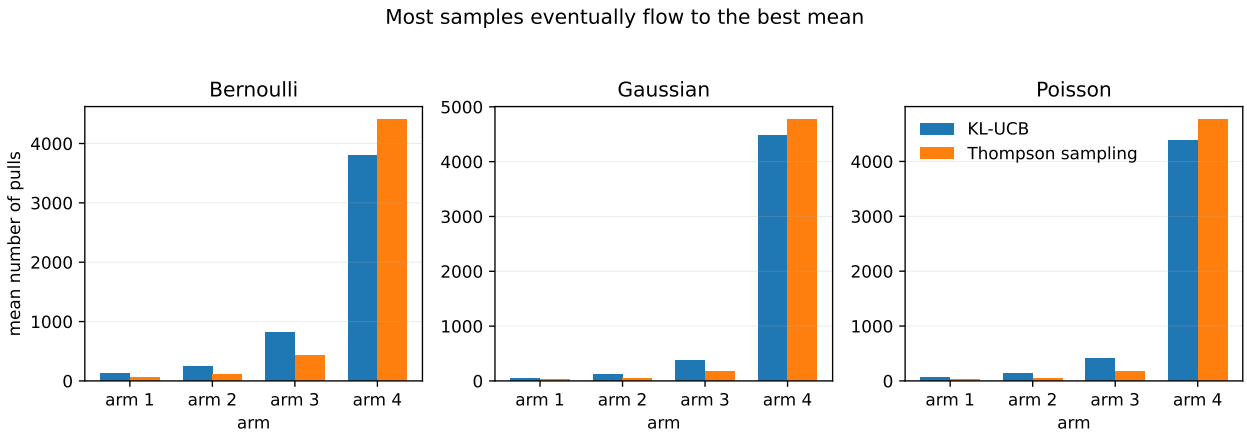


Figure 11.2: Both methods eventually allocate most observations to the arm with the largest mean.

Table 11.1: Simulation results at $T = 5000$. Standard errors are across 240 runs.

Family	Algorithm	Final pseudo-regret	Pulls of best arm
Bernoulli	KL-UCB	92.98 ± 1.32	3806.2
Bernoulli	Thompson sampling	44.66 ± 1.49	4402.2
Gaussian	KL-UCB	54.74 ± 0.76	4470.0
Gaussian	Thompson sampling	23.74 ± 0.67	4766.5
Poisson	KL-UCB	265.43 ± 3.69	4388.0
Poisson	Thompson sampling	101.14 ± 3.01	4761.7

The numerical ranking is not a universal benchmark. It depends on the selected instances, priors, and exploration schedule. The experiment is designed to demonstrate a structural point: one decision loop can operate across reward types when the model supplies the right sufficient statistic, KL divergence, and posterior update.

11.1 What the code makes visible

The Bernoulli policy stores successes and failures. The Gaussian policy stores a count and reward sum. The Poisson policy stores a count and event total. In all three cases, the posterior or confidence index is computed from those same two running quantities.

This is the computational meaning of the exponential family:

$$\boxed{\text{raw history} \longrightarrow (N_a, S_a) \longrightarrow \text{evidence or posterior} \longrightarrow \text{next action.}}$$

Chapter 12 What the Definition Does Not Say

12.1 It does not say that every parameter is natural

The reported mean and the natural parameter may be very different:

$$\eta = \log \frac{p}{1-p}, \qquad \eta = \frac{\mu}{\sigma^2}, \qquad \eta = \log \lambda.$$

The natural coordinate simplifies likelihood algebra. The mean coordinate simplifies decisions and regret statements. Good proofs move between them deliberately.

12.2 It does not say that the base measure is irrelevant

The factor $h(x)$ cancels in likelihood ratios between members of the same family, but it is part of the distribution. For Poisson rewards, $1/x!$ distinguishes the model from other count models. For Gaussian rewards, the $e^{-x^2/(2\sigma^2)}$ term carries the fixed shape of the noise.

12.3 It does not say that variance is constant

The variance is

$$A''(\eta).$$

For Bernoulli rewards it is $p(1-p)$; for Poisson rewards it is λ; for a known-variance Gaussian it is constant. A distribution-aware algorithm uses this changing curvature instead of pretending all arms have the same noise geometry.

12.4 It does not automatically give anytime validity

The fixed-n Chernoff bound in (8.2) is not automatically valid after arbitrary repeated peeking. The filtration and martingale tools from the previous chapter are still needed to construct time-uniform confidence sequences.

12.5 It does not remove modeling risk

A clean exponential-family likelihood may still be wrong. Clicks may be delayed, counts may be overdispersed, Gaussian noise may be heavy-tailed, and reward distributions may drift. The framework clarifies the consequences of a model; it does not certify that the model matches reality.

Chapter 13 What to Carry Forward

An exponential family is a compact statistical machine.

The density has the form

$$p_\eta(x) = h(x)e^{\eta T(x) - A(\eta)}.$$

The log-partition function normalizes the model:

$$A(\eta) = \log \int h(x)e^{\eta T(x)} \, \mathrm{d}\nu(x).$$

Its first two derivatives give the mean and variance:

$$A'(\eta) = \mathbb{E}_\eta[T(X)], \qquad A''(\eta) = \mathrm{Var}_\eta(T(X)).$$

A sample is compressed to

$$S_n = \sum_{i=1}^{n} T(X_i).$$

Maximum likelihood matches the empirical statistic to the model mean:

$$A'(\widehat{\eta}_n) = \frac{S_n}{n}.$$

Conjugate Bayes adds counts and sufficient statistics:

$$(\xi, \nu) \leftarrow (\xi + S_n, \nu + n).$$

KL divergence is the convexity gap of A:

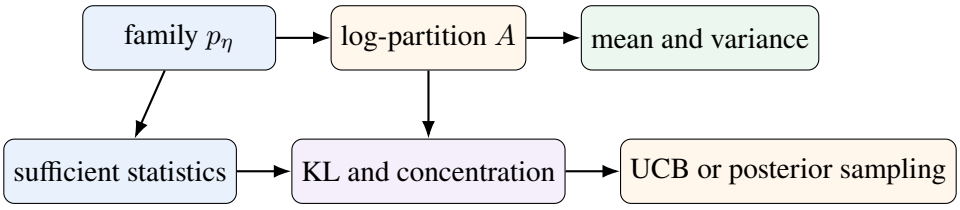
$$\mathrm{KL}(P_\eta \| P_\zeta) = A(\zeta) - A(\eta) - A'(\eta)(\zeta - \eta).$$

Concentration follows from the shifted log-partition function:

$$\log \mathbb{E}_\eta[e^{\lambda T(X)}] = A(\eta + \lambda) - A(\eta).$$

And an adaptive bandit needs only one likelihood ledger per arm:

$$(N_a, S_a).$$



This is why exponential families are a natural language for bandit theory. They do not make every model identical. They reveal which calculations are identical, which quantities remain model-specific, and where statistical evidence enters the decision rule.

Appendix A. Formula Sheet

Core formulas.

Object	Formula
Canonical family	$p_\eta(x) = h(x)e^{\eta T(x) - A(\eta)}$
Log-partition	$A(\eta) = \log \int h(x) e^{\eta T(x)} \, \mathrm{d}\nu(x)$
Mean statistic	$A'(\eta) = \mathbb{E}_\eta[T(X)]$
Variance statistic	$A''(\eta) = \mathrm{Var}_\eta(T(X))$
Vector mean	$\nabla A(\boldsymbol{\eta}) = \mathbb{E}_{\boldsymbol{\eta}}[\boldsymbol{T}(X)]$
Vector covariance	$\nabla^2 A(\boldsymbol{\eta}) = \mathrm{Cov}_{\boldsymbol{\eta}}(\boldsymbol{T}(X))$
Joint sufficient statistic	$S_n = \sum_{i=1}^n T(X_i)$
MLE equation	$A'(\widehat{\eta}_n) = S_n/n$
Conjugate prior	$\pi(\eta \mid \xi, \nu) \propto e^{\xi\eta - \nu A(\eta)}$
Conjugate update	$(\xi, \nu) \mapsto (\xi + S_n, \nu + n)$
Family KL	$\mathrm{KL}(P_\eta \| P_\zeta) = A(\zeta) - A(\eta) - A'(\eta)(\zeta - \eta)$
Fisher information	$I(\eta) = A''(\eta)$
Moment-generating function	$\mathbb{E}_\eta[e^{\lambda T(X)}] = e^{A(\eta+\lambda) - A(\eta)}$
Chernoff exponent	$\mathbb{P}_\eta(\overline{T}_n \geq x) \leq e^{-n \, \mathrm{KL}(P_{\eta_x} \| P_\eta)}$
Bandit ledger	$N_a(t), \; S_a(t) = \sum_{s \leq t : A_s = a} T(X_s)$
KL-UCB index	$\sup\{q : N_a d(\widehat{\mu}_a, q) \leq \beta(t)\}$

Appendix B. Notation Table

Notation.

Symbol	Meaning
X	one observation or reward
$T(X)$	canonical statistic of one observation
$h(x)$	base density or mass independent of the parameter
η	natural parameter
$\mathcal{H}$	natural-parameter space
$A(\eta)$	log-partition function
$\mu = A'(\eta)$	mean parameter for the statistic
S_n	sum of sufficient statistics through n observations
ξ, ν	conjugate-prior hyperparameters
A^*	convex conjugate of A
$d(\mu, q)$	family KL divergence in mean coordinates
A_t	arm selected at round t
$N_a(t)$	number of pulls of arm a by time t
$S_a(t)$	sufficient-statistic total for arm a
$U_a(t)$	upper confidence index

Appendix C. Minimal Implementation Notes

The supplied Python script reproduces all figures and tables. A few details matter in numerical work:

1. Clip Bernoulli probabilities away from exactly 0 and 1 before taking logarithms, while preserving the limiting convention $0 \log 0 = 0$.
2. Use shape-rate consistently for Gamma distributions. NumPy and SciPy accept a scale, which is the reciprocal of the rate.
3. Invert one-sided Bernoulli or Poisson KL balls by monotone bisection. A fixed iteration count gives predictable numerical cost.
4. Initialize every arm before dividing by $N_a(t)$.
5. Treat the simulation table as an audit of the implementation, not a general ranking of algorithms.

Further Reading

Barndorff-Nielsen's monograph develops the statistical theory of exponential families in depth [2]. Wainwright and Jordan give the modern convex-analytic view that connects exponential families, marginal polytopes, and variational inference [12]. MacKay's Cambridge text is an unusually readable route through likelihood, Bayesian updating, and information geometry [11]; Bishop's treatment from Microsoft Research Cambridge is a useful machine-learning companion [3]. Lattimore and Szepesvari place these ideas directly inside bandit theory [10]. For Thompson sampling in one-dimensional exponential families, see Korda, Kaufmann, and Munos [8] and the finite-time development of Jin and collaborators [7].

Appendix D. Full Experiment Script

```
"""Reproducible experiments for
'Exponential Families: The Natural Language of Modern Bandit Models'.

The script creates:
  - log_partition_map.pdf/png
  - sufficient_statistic_likelihood.pdf/png
  - conjugate_updates.pdf/png
  - rate_functions.pdf/png
  - exponential_family_regret.pdf/png
  - exponential_family_pull_counts.pdf/png
  - exponential_families_results.csv

All simulations use fixed random seeds.
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
import math

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.special import expit, gammaln
from scipy.stats import beta as beta_dist
from scipy.stats import gamma as gamma_dist
from scipy.stats import norm


OUT = Path(__file__).resolve().parent
SEED = 20260620


@dataclass(frozen=True)
class Config:
    horizon: int = 5000
    runs: int = 240
    binary_steps: int = 22

    bernoulli_means: tuple[float, ...] = (0.18, 0.24, 0.31, 0.36)
    gaussian_means: tuple[float, ...] = (0.00, 0.10, 0.18, 0.25)
    gaussian_sigma: float = 0.35
    poisson_means: tuple[float, ...] = (1.70, 2.00, 2.30, 2.60)


def bernoulli_kl(p: np.ndarray | float, q: np.ndarray | float) -> np.ndarray:
    p_arr = np.asarray(p, dtype=float)
    q_arr = np.asarray(q, dtype=float)
    q_safe = np.clip(q_arr, 1e-14, 1.0 - 1e-14)
    p_safe = np.clip(p_arr, 1e-14, 1.0 - 1e-14)
    term1 = np.where(p_arr > 0.0, p_safe * np.log(p_safe / q_safe), 0.0)
    term2 = np.where(
        p_arr < 1.0,
        (1.0 - p_safe) * np.log((1.0 - p_safe) / (1.0 - q_safe)),
        0.0,
    )
```

```
    return term1 + term2


def poisson_kl(p: np.ndarray | float, q: np.ndarray | float) -> np.ndarray:
    p_arr = np.asarray(p, dtype=float)
    q_arr = np.asarray(q, dtype=float)
    q_safe = np.clip(q_arr, 1e-14, None)
    p_safe = np.clip(p_arr, 1e-14, None)
    return np.where(p_arr > 0.0, p_safe * np.log(p_safe / q_safe), 0.0) + q_safe - p_arr


def bernoulli_kl_upper(emp: np.ndarray, counts: np.ndarray, beta: float, steps: int) -> np.ndarray:
    lo = emp.copy()
    hi = np.ones_like(emp)
    for _ in range(steps):
        mid = (lo + hi) / 2.0
        feasible = counts * bernoulli_kl(emp, mid) <= beta
        lo = np.where(feasible, mid, lo)
        hi = np.where(feasible, hi, mid)
    return lo


def poisson_kl_upper(emp: np.ndarray, counts: np.ndarray, beta: float, steps: int) -> np.ndarray:
    radius = beta / counts
    lo = np.maximum(emp, 0.0)
    hi = np.maximum(1.0, emp + 2.0 * np.sqrt(2.0 * (emp + 1.0) * radius) + 4.0 * radius + 2.0)
    # The bracket above is deliberately generous for the means used here.
    for _ in range(steps):
        mid = (lo + hi) / 2.0
        feasible = counts * poisson_kl(emp, mid) <= beta
        lo = np.where(feasible, mid, lo)
        hi = np.where(feasible, hi, mid)
    return lo


def plot_log_partition_map() -> pd.DataFrame:
    eta_b = np.linspace(-5.0, 5.0, 500)
    A_b = np.logaddexp(0.0, eta_b)
    m_b = expit(eta_b)
    v_b = m_b * (1.0 - m_b)

    eta_g = np.linspace(-3.0, 3.0, 500)
    A_g = 0.5 * eta_g**2
    m_g = eta_g
    v_g = np.ones_like(eta_g)

    eta_p = np.linspace(-2.5, 1.7, 500)
    A_p = np.exp(eta_p)
    m_p = np.exp(eta_p)
    v_p = np.exp(eta_p)

    fig, axes = plt.subplots(3, 3, figsize=(10.0, 8.0))
    families = [
        ("Bernoulli", eta_b, A_b, m_b, v_b),
        ("Gaussian, $\\sigma^2=1$", eta_g, A_g, m_g, v_g),
        ("Poisson", eta_p, A_p, m_p, v_p),
    ]
    for row, (name, eta, A, mean, var) in enumerate(families):
        axes[row, 0].plot(eta, A, linewidth=1.9)
        axes[row, 0].set_ylabel(name)
        axes[row, 1].plot(eta, mean, linewidth=1.9)
```

```
        axes[row, 2].plot(eta, var, linewidth=1.9)
        for col in range(3):
            axes[row, col].axhline(0.0, linewidth=0.6, alpha=0.45)
            axes[row, col].grid(True, linewidth=0.3, alpha=0.25)
            if row == 2:
                axes[row, col].set_xlabel(r"natural parameter $\eta$")
    axes[0, 0].set_title(r"$A(\eta)$")
    axes[0, 1].set_title(r"$A'(\eta)=\mathbb{E}[T(X)]$")
    axes[0, 2].set_title(r"$A''(\eta)=\mathrm{Var}(T(X))$")
    fig.suptitle("One function stores normalization, mean, and variance", y=1.01)
    fig.tight_layout()
    fig.savefig(OUT / "log_partition_map.pdf", bbox_inches="tight")
    fig.savefig(OUT / "log_partition_map.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    return pd.DataFrame(
        [
            {"experiment": "log_partition", "quantity": "Bernoulli mean at eta=0", "value": 0.5},
            {"experiment": "log_partition", "quantity": "Bernoulli variance at eta=0", "value": 0.25},
            {"experiment": "log_partition", "quantity": "Gaussian variance for sigma2=1", "value": 1.0},
            {"experiment": "log_partition", "quantity": "Poisson mean at eta=log(2.5)", "value": 2.5},
        ]
    )


def plot_sufficient_statistic_likelihood() -> pd.DataFrame:
    n = 12
    sequences = {
        "sequence A: 7 clicks": np.array([1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1]),
        "sequence B: same 7 clicks": np.array([0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1]),
        "sequence C: 4 clicks": np.array([1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0]),
    }
    p = np.linspace(0.01, 0.99, 600)

    fig, ax = plt.subplots(figsize=(7.3, 4.55))
    rows: list[dict[str, float | str]] = []
    styles = ["-", "--", "-."]
    for (label, seq), style in zip(sequences.items(), styles):
        s = int(seq.sum())
        log_lik = s * np.log(p) + (n - s) * np.log(1.0 - p)
        log_lik -= log_lik.max()
        ax.plot(p, np.exp(log_lik), linestyle=style, linewidth=2.0, label=label)
        rows.append({"experiment": "sufficiency", "quantity": f"success count for {label}", "value": float(s)})
    ax.set_xlabel(r"candidate click probability $p$")
    ax.set_ylabel("relative likelihood")
    ax.set_title("Order disappears: the likelihood remembers only the success count")
    ax.legend(frameon=False)
    ax.grid(True, linewidth=0.3, alpha=0.25)
    fig.tight_layout()
    fig.savefig(OUT / "sufficient_statistic_likelihood.pdf", bbox_inches="tight")
    fig.savefig(OUT / "sufficient_statistic_likelihood.png", dpi=220, bbox_inches="tight")
    plt.close(fig)
    return pd.DataFrame(rows)


def plot_conjugate_updates() -> pd.DataFrame:
    fig, axes = plt.subplots(1, 3, figsize=(10.2, 3.55))
    rows: list[dict[str, float | str]] = []

    # Beta-Bernoulli
    p = np.linspace(0.001, 0.999, 600)
```

```python
    a0, b0 = 1.0, 1.0
    successes, failures = 14, 6
    a1, b1 = a0 + successes, b0 + failures
    axes[0].plot(p, beta_dist.pdf(p, a0, b0), linewidth=1.7, label="prior")
    axes[0].plot(p, beta_dist.pdf(p, a1, b1), linewidth=2.0, label="posterior")
    axes[0].axvline(successes / (successes + failures), linewidth=0.9, linestyle="--")
    axes[0].set_title("Bernoulli mean")
    axes[0].set_xlabel(r"$p$")
    axes[0].set_ylabel("density")
    axes[0].legend(frameon=False)
    rows.append({"experiment": "conjugacy", "quantity": "Beta posterior mean", "value": a1 / (a1 + b1)})

    # Gaussian known variance
    mu = np.linspace(-0.8, 1.2, 600)
    prior_mean, prior_sd = 0.0, 0.7
    sigma = 0.5
    n, xbar = 12, 0.45
    prior_precision = 1.0 / prior_sd**2
    data_precision = n / sigma**2
    post_var = 1.0 / (prior_precision + data_precision)
    post_mean = post_var * (prior_precision * prior_mean + data_precision * xbar)
    axes[1].plot(mu, norm.pdf(mu, prior_mean, prior_sd), linewidth=1.7, label="prior")
    axes[1].plot(mu, norm.pdf(mu, post_mean, math.sqrt(post_var)), linewidth=2.0, label="posterior")
    axes[1].axvline(xbar, linewidth=0.9, linestyle="--")
    axes[1].set_title("Gaussian mean")
    axes[1].set_xlabel(r"$\mu$")
    axes[1].legend(frameon=False)
    rows.append({"experiment": "conjugacy", "quantity": "Gaussian posterior mean", "value": post_mean})

    # Gamma-Poisson (shape-rate)
    lam = np.linspace(0.001, 5.5, 600)
    shape0, rate0 = 1.0, 1.0
    n_pois, total = 12, 30
    shape1, rate1 = shape0 + total, rate0 + n_pois
    axes[2].plot(lam, gamma_dist.pdf(lam, a=shape0, scale=1.0 / rate0), linewidth=1.7, label="prior")
    axes[2].plot(lam, gamma_dist.pdf(lam, a=shape1, scale=1.0 / rate1), linewidth=2.0, label="posterior")
    axes[2].axvline(total / n_pois, linewidth=0.9, linestyle="--")
    axes[2].set_title("Poisson rate")
    axes[2].set_xlabel(r"$\lambda$")
    axes[2].legend(frameon=False)
    rows.append({"experiment": "conjugacy", "quantity": "Gamma posterior mean", "value": shape1 / rate1})

    for ax in axes:
        ax.grid(True, linewidth=0.3, alpha=0.25)
    fig.suptitle("Conjugacy is a bookkeeping rule: old pseudo-data plus new data", y=1.03)
    fig.tight_layout()
    fig.savefig(OUT / "conjugate_updates.pdf", bbox_inches="tight")
    fig.savefig(OUT / "conjugate_updates.png", dpi=220, bbox_inches="tight")
    plt.close(fig)
    return pd.DataFrame(rows)


def plot_rate_functions() -> pd.DataFrame:
    fig, axes = plt.subplots(1, 3, figsize=(10.2, 3.55))
    rows: list[dict[str, float | str]] = []

    mu_b = 0.35
    x_b = np.linspace(0.005, 0.995, 600)
    rate_b = bernoulli_kl(x_b, mu_b)
    axes[0].plot(x_b, rate_b, linewidth=2.0)
    axes[0].axvline(mu_b, linewidth=0.9, linestyle="--")
```

```
    axes[0].set_title("Bernoulli")
    axes[0].set_xlabel(r"sample mean $x$")
    axes[0].set_ylabel("rate function")

    mu_g, sigma = 0.25, 0.35
    x_g = np.linspace(-0.8, 1.3, 600)
    rate_g = (x_g - mu_g) ** 2 / (2.0 * sigma**2)
    axes[1].plot(x_g, rate_g, linewidth=2.0)
    axes[1].axvline(mu_g, linewidth=0.9, linestyle="--")
    axes[1].set_title("Gaussian")
    axes[1].set_xlabel(r"sample mean $x$")

    mu_p = 2.6
    x_p = np.linspace(0.01, 6.5, 600)
    rate_p = poisson_kl(x_p, mu_p)
    axes[2].plot(x_p, rate_p, linewidth=2.0)
    axes[2].axvline(mu_p, linewidth=0.9, linestyle="--")
    axes[2].set_title("Poisson")
    axes[2].set_xlabel(r"sample mean $x$")

    for ax in axes:
        ax.set_ylim(bottom=0.0)
        ax.grid(True, linewidth=0.3, alpha=0.25)
    fig.suptitle("Different reward models, the same concentration story", y=1.03)
    fig.tight_layout()
    fig.savefig(OUT / "rate_functions.pdf", bbox_inches="tight")
    fig.savefig(OUT / "rate_functions.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    rows.extend(
        [
            {"experiment": "rate_function", "quantity": "Bernoulli rate at x=0.45", "value": float(bernoulli_kl
(0.45, mu_b))},
            {"experiment": "rate_function", "quantity": "Gaussian rate at x=0.45", "value": float((0.45 - mu_g)
** 2 / (2.0 * sigma**2))},
            {"experiment": "rate_function", "quantity": "Poisson rate at x=3.2", "value": float(poisson_kl(3.2,
mu_p))},
        ]
    )
    return pd.DataFrame(rows)


def _beta_time(t: int) -> float:
    if t <= 2:
        return 1.0
    return math.log(t) + 3.0 * math.log(max(math.log(t), 1.0))


def simulate_bernoulli(cfg: Config, algorithm: str, seed: int) -> tuple[np.ndarray, np.ndarray]:
    rng = np.random.default_rng(seed)
    means = np.asarray(cfg.bernoulli_means, dtype=float)
    runs, horizon, k = cfg.runs, cfg.horizon, means.size
    counts = np.zeros((runs, k), dtype=float)
    sums = np.zeros((runs, k), dtype=float)
    regrets = np.zeros((runs, horizon), dtype=float)
    best = float(means.max())
    rr = np.arange(runs)

    for a in range(k):
        reward = (rng.random(runs) < means[a]).astype(float)
        counts[:, a] += 1.0
```

```
        sums[:, a] += reward
        regrets[:, a] = (best - means[a]) + (regrets[:, a - 1] if a > 0 else 0.0)

    for t in range(k, horizon):
        emp = sums / counts
        if algorithm == "KL-UCB":
            index = bernoulli_kl_upper(emp, counts, _beta_time(t + 1), cfg.binary_steps)
        elif algorithm == "Thompson sampling":
            index = rng.beta(1.0 + sums, 1.0 + counts - sums)
        else:
            raise ValueError(f"unknown algorithm: {algorithm}")
        action = np.argmax(index, axis=1)
        reward = (rng.random(runs) < means[action]).astype(float)
        counts[rr, action] += 1.0
        sums[rr, action] += reward
        regrets[:, t] = regrets[:, t - 1] + best - means[action]
    return regrets, counts


def simulate_gaussian(cfg: Config, algorithm: str, seed: int) -> tuple[np.ndarray, np.ndarray]:
    rng = np.random.default_rng(seed)
    means = np.asarray(cfg.gaussian_means, dtype=float)
    sigma = cfg.gaussian_sigma
    runs, horizon, k = cfg.runs, cfg.horizon, means.size
    counts = np.zeros((runs, k), dtype=float)
    sums = np.zeros((runs, k), dtype=float)
    regrets = np.zeros((runs, horizon), dtype=float)
    best = float(means.max())
    rr = np.arange(runs)

    for a in range(k):
        reward = rng.normal(means[a], sigma, size=runs)
        counts[:, a] += 1.0
        sums[:, a] += reward
        regrets[:, a] = (best - means[a]) + (regrets[:, a - 1] if a > 0 else 0.0)

    prior_mean, prior_var = 0.0, 1.0
    for t in range(k, horizon):
        emp = sums / counts
        if algorithm == "KL-UCB":
            index = emp + np.sqrt(2.0 * sigma**2 * _beta_time(t + 1) / counts)
        elif algorithm == "Thompson sampling":
            precision = 1.0 / prior_var + counts / sigma**2
            post_var = 1.0 / precision
            post_mean = post_var * (prior_mean / prior_var + sums / sigma**2)
            index = rng.normal(post_mean, np.sqrt(post_var))
        else:
            raise ValueError(f"unknown algorithm: {algorithm}")
        action = np.argmax(index, axis=1)
        reward = rng.normal(means[action], sigma)
        counts[rr, action] += 1.0
        sums[rr, action] += reward
        regrets[:, t] = regrets[:, t - 1] + best - means[action]
    return regrets, counts


def simulate_poisson(cfg: Config, algorithm: str, seed: int) -> tuple[np.ndarray, np.ndarray]:
    rng = np.random.default_rng(seed)
    means = np.asarray(cfg.poisson_means, dtype=float)
    runs, horizon, k = cfg.runs, cfg.horizon, means.size
    counts = np.zeros((runs, k), dtype=float)
```

```
    sums = np.zeros((runs, k), dtype=float)
    regrets = np.zeros((runs, horizon), dtype=float)
    best = float(means.max())
    rr = np.arange(runs)

    for a in range(k):
        reward = rng.poisson(means[a], size=runs).astype(float)
        counts[:, a] += 1.0
        sums[:, a] += reward
        regrets[:, a] = (best - means[a]) + (regrets[:, a - 1] if a > 0 else 0.0)

    for t in range(k, horizon):
        emp = sums / counts
        if algorithm == "KL-UCB":
            index = poisson_kl_upper(emp, counts, _beta_time(t + 1), cfg.binary_steps)
        elif algorithm == "Thompson sampling":
            index = rng.gamma(shape=1.0 + sums, scale=1.0 / (1.0 + counts))
        else:
            raise ValueError(f"unknown algorithm: {algorithm}")
        action = np.argmax(index, axis=1)
        reward = rng.poisson(means[action]).astype(float)
        counts[rr, action] += 1.0
        sums[rr, action] += reward
        regrets[:, t] = regrets[:, t - 1] + best - means[action]
    return regrets, counts


def run_bandit_experiments(cfg: Config) -> pd.DataFrame:
    families = {
        "Bernoulli": (simulate_bernoulli, np.asarray(cfg.bernoulli_means)),
        "Gaussian": (simulate_gaussian, np.asarray(cfg.gaussian_means)),
        "Poisson": (simulate_poisson, np.asarray(cfg.poisson_means)),
    }
    algorithms = ("KL-UCB", "Thompson sampling")
    curves: dict[tuple[str, str], np.ndarray] = {}
    counts_map: dict[tuple[str, str], np.ndarray] = {}
    rows: list[dict[str, float | str]] = []

    for f_idx, (family, (simulator, means)) in enumerate(families.items()):
        for a_idx, algorithm in enumerate(algorithms):
            regrets, counts = simulator(cfg, algorithm, SEED + 1000 * f_idx + 100 * a_idx)
            curves[(family, algorithm)] = regrets.mean(axis=0)
            counts_map[(family, algorithm)] = counts.mean(axis=0)
            final = regrets[:, -1]
            rows.extend(
                [
                    {"experiment": "bandit", "family": family, "algorithm": algorithm, "quantity": "mean final
     pseudo-regret", "value": float(final.mean())},
                    {"experiment": "bandit", "family": family, "algorithm": algorithm, "quantity": "standard
     error final pseudo-regret", "value": float(final.std(ddof=1) / math.sqrt(cfg.runs))},
                    {"experiment": "bandit", "family": family, "algorithm": algorithm, "quantity": "mean pulls of
      best arm", "value": float(counts[:, int(np.argmax(means))].mean())},
                ]
            )

    time = np.arange(1, cfg.horizon + 1)
    fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.55), sharex=True)
    for ax, family in zip(axes, families.keys()):
        for algorithm in algorithms:
            ax.plot(time, curves[(family, algorithm)], linewidth=1.9, label=algorithm)
        ax.set_title(family)
```

```
        ax.set_xlabel("round")
        ax.grid(True, linewidth=0.3, alpha=0.25)
    axes[0].set_ylabel("mean pseudo-regret")
    axes[-1].legend(frameon=False)
    fig.suptitle("The policy skeleton stays the same; the family supplies the right evidence scale", y=1.03)
    fig.tight_layout()
    fig.savefig(OUT / "exponential_family_regret.pdf", bbox_inches="tight")
    fig.savefig(OUT / "exponential_family_regret.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.55))
    x = np.arange(4)
    width = 0.36
    for ax, (family, (_, means)) in zip(axes, families.items()):
        for j, algorithm in enumerate(algorithms):
            ax.bar(x + (j - 0.5) * width, counts_map[(family, algorithm)], width=width, label=algorithm)
        ax.set_title(family)
        ax.set_xticks(x, [f"arm {j+1}" for j in x])
        ax.set_xlabel("arm")
        ax.grid(True, axis="y", linewidth=0.3, alpha=0.25)
    axes[0].set_ylabel("mean number of pulls")
    axes[-1].legend(frameon=False)
    fig.suptitle("Most samples eventually flow to the best mean", y=1.03)
    fig.tight_layout()
    fig.savefig(OUT / "exponential_family_pull_counts.pdf", bbox_inches="tight")
    fig.savefig(OUT / "exponential_family_pull_counts.png", dpi=220, bbox_inches="tight")
    plt.close(fig)

    return pd.DataFrame(rows)


def main() -> None:
    cfg = Config()
    frames = [
        plot_log_partition_map(),
        plot_sufficient_statistic_likelihood(),
        plot_conjugate_updates(),
        plot_rate_functions(),
        run_bandit_experiments(cfg),
    ]
    results = pd.concat(frames, ignore_index=True, sort=False)
    results.to_csv(OUT / "exponential_families_results.csv", index=False)
    print(results.to_string(index=False))


if __name__ == "__main__":
    main()
```

Bibliography

[1] Arindam Banerjee et al. “Clustering with Bregman Divergences”. In: *Journal of Machine Learning Research* 6 (2005), pp. 1705–1749.

[2] Ole E. Barndorff-Nielsen. *Information and Exponential Families in Statistical Theory*. Wiley, 1978.

[3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[4] Lawrence D. Brown. *Fundamentals of Statistical Exponential Families with Applications in Statistical Decision Theory*. Institute of Mathematical Statistics, 1986.

[5] Aurélien Garivier and Olivier Cappé. “The KL-UCB Algorithm for Bounded Stochastic Bandits and Beyond”. In: *Proceedings of the 24th Annual Conference on Learning Theory*. 2011.

[6] Z. Ghahramani. “Probabilistic machine learning and artificial intelligence”. In: *Nature* 521 (2015), pp. 452–459.

[7] T. Jin et al. *Finite-time regret of Thompson sampling algorithms for exponential family multi-armed bandits*. arXiv:2206.03520. 2022.

[8] Nathaniel Korda, Emilie Kaufmann, and Rémi Munos. “Thompson Sampling for 1-Dimensional Exponential Family Bandits”. In: *Advances in Neural Information Processing Systems*. 2013.

[9] T. L. Lai and H. Robbins. “Asymptotically efficient adaptive allocation rules”. In: *Advances in Applied Mathematics* 6.1 (1985), pp. 4–22.

[10] T. Lattimore and C. Szepesvari. *Bandit Algorithms*. Cambridge University Press, 2020.

[11] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[12] Martin J. Wainwright and Michael I. Jordan. “Graphical Models, Exponential Families, and Variational Inference”. In: *Foundations and Trends in Machine Learning* 1.1–2 (2008), pp. 1–305.